

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Conception et développement d'une plateforme d'inventaire logistique multi-canal

Rapport de Projet de Fin d'Études

Réalisé par : **Harrabi Yosri**

Spécialité : Génie Logiciel et Systèmes d'Information

Encadrant professionnel : M. Ferchichi Marwen — VECTORYS Tunisie

Encadrante académique : Mme Zrelli Siwar — TEK-UP

Année universitaire 2025 — 2026

Dédicace

[À compléter par l'étudiant]

« À mes parents, pour leur soutien indéfectible tout au long de mon parcours académique. »

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet de fin d'études. Je remercie tout d'abord **M. Ferchichi Marwen**, mon encadrant professionnel au sein de **VECTORYS Tunisie**, pour sa disponibilité, sa vision métier et ses orientations précieuses tout au long du stage. Mes remerciements s'adressent également à **Mme Zrelli Siwar**, mon encadrante académique à TEK-UP, pour le suivi rigoureux et les recommandations méthodologiques. Enfin, je remercie l'équipe VECTORYS et l'ensemble du corps enseignant de TEK-UP pour l'environnement de travail propice à l'apprentissage et à l'innovation.

Résumé

Ce projet de fin d'études porte sur la conception et le développement de **Vectorys Inventaire**, une plateforme multi-canal dédiée à la digitalisation des opérations d'inventaire physique en entrepôt. La solution articule une API REST ASP.NET Core 9, une application web Angular 19 de supervision et une application mobile Flutter pour le comptage terrain. Elle couvre l'ensemble du cycle métier : import Excel du stock théorique, assignation stricte des agents, scan des unités de stock avec clôture automatique, mode hors ligne, détection des écarts, workflows de correction (branches A et B) et tableaux de bord KPI. Le projet a été mené selon Scrum en dix sprints, tous achevés, au sein de VECTORYS Tunisie.

Mots-clés : inventaire logistique, multi-canal, ASP.NET Core, Flutter, Angular, Scrum, entrepôt.

Abstract

This graduation project focuses on designing and developing **Vectorys Inventaire**, a multi-channel platform for digitizing physical warehouse inventory operations. The solution combines an ASP.NET Core 9 REST API, an Angular 19 supervisory web application, and a Flutter mobile app for field counting. It covers the full business cycle: Excel import of theoretical stock, strict agent assignment, barcode scanning with automatic location closure, offline mode, discrepancy detection, correction workflows (branches A and B), and KPI dashboards. The project was conducted using Scrum across ten completed sprints at VECTORYS Tunisia.

Keywords: logistics inventory, multi-channel, ASP.NET Core, Flutter, Angular, Scrum, warehouse.

Table des matières — Lot 1

Introduction générale

Chapitre 1 — Cadre du projet

1.1 Cadre général du projet

1.2 Étude de l'existant

1.3 Problématique

1.4 Solution proposée

1.5 Méthodologie de gestion de projet

Chapitre 2 — Analyse et spécification des besoins

2.1 Identification des acteurs

2.2 Analyse des besoins fonctionnels

2.3 Besoins non fonctionnels

2.4 Diagramme de cas d'utilisation

2.5 Scénario principal de comptage C1

2.6 Backlog produit et user stories

2.7 Matrice de traçabilité

Introduction générale

La gestion des stocks constitue un enjeu stratégique pour les opérateurs logistiques. Une discordance entre le stock théorique enregistré dans les systèmes d'information et la réalité physique observée sur le terrain engendre des pertes financières, des ruptures de service et une dégradation de la relation client. L'inventaire physique demeure une pratique incontournable, malgré sa lourdeur organisationnelle.

VECTORYS Tunisie fait face à cette réalité au quotidien. Les inventaires y étaient pilotés à l'aide d'outils hétérogènes (Excel, terminaux dédiés, ERP hérités) ne facilitant ni la coordination terrain, ni le traitement structuré des écarts. C'est dans ce cadre que s'inscrit le présent projet, intitulé « Conception et développement d'une plateforme d'inventaire logistique multi-canal ».

La plateforme **Vectorys Inventaire** repose sur trois composants : une API REST ASP.NET Core 9 (Clean Architecture, CQRS/MediatR), une application web Angular 19 de supervision et une application mobile Flutter pour le scan terrain. SQL Server centralise les données ; SignalR assure le temps réel ; SQLite local permet le mode hors ligne. Le projet a été mené selon Scrum en dix sprints, tous achevés.

Ce rapport décrit progressivement le travail réalisé : le chapitre 1 présente le cadre du projet ; le chapitre 2 analyse les besoins ; le chapitre 3 détaille la conception ; le chapitre 4 expose la réalisation par sprint ; le chapitre 5 aborde les tests et la validation.

Chapitre 1 — Cadre du projet

Ce chapitre situe le projet dans son environnement organisationnel et métier, formule la problématique, décrit la solution retenue et expose la méthodologie Scrum adoptée.

1.1 Cadre général du projet

1.1.1 Présentation de l'organisme d'accueil

VECTORYS Tunisie est une entreprise de logistique et supply chain implantée à Radès. Elle assure la gestion d'entrepôts, la préparation de commandes et le contrôle des flux pour ses clients. Les inventaires tournants ou annuels mobilisent superviseurs et agents terrain ; leur précision conditionne directement la fiabilité des opérations de picking et d'expédition.

Le stage s'est déroulé sous l'encadrement de M. Ferchichi Marwen (VECTORYS) et de Mme Zrelli Siwar (TEK-UP), dans le cadre du diplôme d'ingénieur en Génie Logiciel et Systèmes d'Information.

1.1.2 Contexte et objectifs du projet

Les objectifs principaux du projet ont été les suivants :

1. Digitaliser la préparation d'inventaire (création mission, import Excel, passage C1, assignation agents).
2. Outiller le comptage terrain (scan en boucle, quotas US, clôture automatique des emplacements).
3. Assurer traçabilité et résilience (horodatage, mode hors ligne, synchronisation sans doublon).
4. Automatiser la détection des écarts et les workflows de correction (branches A et B).
5. Offrir un pilotage temps réel et des tableaux de bord KPI.

1.2 Étude de l'existant

1.2.1 Pratiques internes

Avant le projet, le processus reposait sur l'extraction Excel depuis l'ERP Sage, la répartition manuelle des zones, une saisie partielle sur terminaux et une consolidation tardive. Les limites identifiées étaient : absence de source de vérité temps réel, risques de doublons, suivi d'avancement difficile, traitement des écarts non standardisé et KPI produits tardivement.

1.2.2 Solutions du marché

Les WMS généralistes et les applications de scan génériques ne couvrent pas la logique métier VECTORYS (passages C1/C2, assignation stricte CodeAgent, workflows d'écarts). Un espace fonctionnel dédié s'est imposé.

1.3 Problématique

Comment concevoir et développer une plateforme d'inventaire logistique multi-canal, fiable et évolutive, capable de synchroniser en temps réel le comptage terrain avec le pilotage superviseur, tout en garantissant la résilience hors connexion et le traitement structuré des écarts ?

1.4 Solution proposée

1.4.1 Architecture générale

Le système suit une architecture three-tier : couche présentation (Angular + Flutter), couche métier (API ASP.NET Core, MediatR, FluentValidation, SignalR) et couche données (SQL Server, EF Core). La figure 1.1 illustre cette organisation.

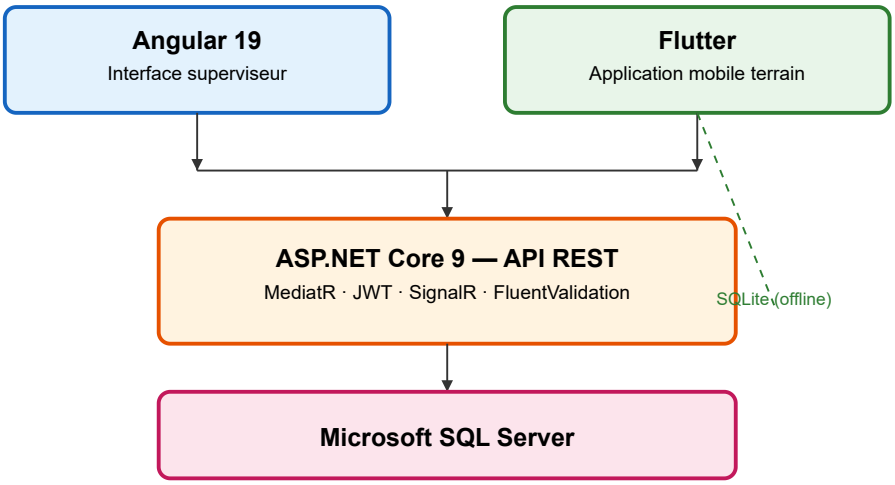


Figure 1.1 — Architecture three-tier de la plateforme Vectorys Inventaire

1.4.2 Flux métier principaux

Cinq flux structurants ont guidé la conception (figure 1.2) :

1. Préparation : import Excel → inventaire → C1 → assignation CodeAgent.
2. Comptage : scan emplacement → boucle US → quota → clôture automatique.
3. Détection d'écarts : comparaison théorique / réel.
4. Résolution : branche A (C2 ciblé) ou branche B (recomptage global).
5. Clôture : export Excel, KPI, analyse par zone client.



Figure 1.2 — Flux métier principal du cycle d'inventaire Vectorys

1.4.3 Stack technologique

Composant	Technologie	Rôle
Backend	ASP.NET Core 9, C#	API REST, SignalR, JWT
ORM	Entity Framework Core 9	SQL Server, migrations
Web	Angular 19, Chart.js	Supervision, KPI
Mobile	Flutter, SQLite	Scan, mode offline
Import/Export	ClosedXML	Fichiers Excel

1.5 Méthodologie de gestion de projet

1.5.1 Choix de la méthode Scrum

Le projet a été mené selon Scrum, méthode agile adaptée aux projets logiciels évolutifs. Des sprints de une à deux semaines ont livré des incréments testables, de la fondation technique au déploiement. La figure 1.3 présente le cadre Scrum appliqué au projet.

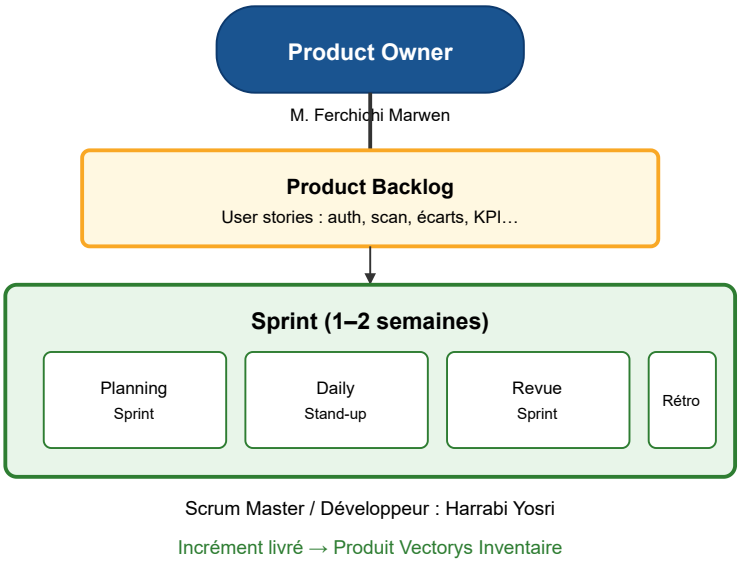


Figure 1.3 — Cadre Scrum et répartition des rôles dans le projet

1.5.2 Rôles et cérémonies

- **Product Owner** : priorisation du backlog, validation métier.
- **Scrum Master / développeur** : organisation des sprints, implémentation, documentation.
- **Cérémonies** : planification, daily stand-up, revue de sprint, rétrospective.

1.5.3 Découpage en sprints

Le tableau ci-dessous résume le découpage fonctionnel et les livrables réalisés. L'ensemble des dix sprints (0 à 9) a été mené à terme. La figure 1.4 en propose une vue chronologique.

Sprint	Objectif principal
0	Fondations : structure solution, base de données et dépôt Git
1	Authentification JWT pour les interfaces web et mobile
2	Création d'inventaire, import Excel et assignation des agents
3	Scan Flutter et SignalR temps réel
4	Mode hors ligne et synchronisation batch
5	Détection et gestion des écarts (branche A)
6	Branche B, clôture et export Excel final
7	Module KPI et analytics (Chart.js)
8	Tests automatisés et documentation Swagger

9	Déploiement IIS, APK terrain et validation end-to-end

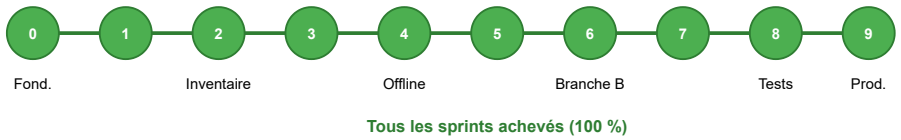


Figure 1.4 — Frise chronologique des sprints 0 à 9 (version finale)

Ce découpage a permis une montée en charge progressive : fondations, métier, résilience offline, analyse des écarts et KPI, puis tests et déploiement. À l'issue du sprint 9, la plateforme couvre l'intégralité du cycle d'inventaire prévu.

Conclusion

Ce chapitre a situé le projet dans son contexte VECTORYS, formulé la problématique et présenté la solution Vectorys Inventaire ainsi que la méthodologie Scrum retenue. Le chapitre suivant détaillera l'analyse des besoins fonctionnels et non fonctionnels, les acteurs et les spécifications du système.

Chapitre 2 — Analyse et spécification des besoins

Ce chapitre traduit les objectifs métier en exigences formalisées. Il identifie les acteurs, détaille les besoins fonctionnels et non fonctionnels, présente le diagramme de cas d'utilisation, le backlog produit et la matrice de traçabilité vers les sprints réalisés.

Introduction

L'analyse des besoins constitue une étape structurante entre le cadrage du projet (chapitre 1) et sa conception technique (chapitre 3). Elle s'appuie sur des entretiens avec l'encadrant professionnel, l'observation des pratiques d'inventaire à VECTORYS Radès et l'analyse des fichiers Excel existants. Les exigences ont été priorisées dans un backlog Scrum et implémentées au fil des dix sprints, tous achevés dans la version finale du produit.

2.1 Identification des acteurs

2.1.1 Acteurs humains

Le système distingue trois rôles applicatifs, matérialisés par l'énumération *RoleEnum* :

Acteur	Rôle technique	Canal	Responsabilités principales
Superviseur	SUPERVISEUR	Application web Angular	Créer les inventaires, importer le stock théorique, suivre l'avancement temps réel, détecter et traiter les écarts, consulter les KPI.
Agent de comptage	AGENT	Application mobile Flutter	S'authentifier via son code personnel, consulter ses emplacements assignés, scanner les unités de stock, synchroniser les données offline.
Administrateur	ADMIN	Application web	Gérer les comptes utilisateurs, disposer des mêmes privilèges de supervision que le superviseur.

L'authentification est dual-mode : l'agent se connecte avec son *CodePersonnel* (ex. AG001), tandis que le superviseur utilise email et mot de passe. Un JWT est émis avec une durée de validité de 8 h (agent) ou 24 h (superviseur/admin), transportant les claims *sub*, *role*, *nom* et *codePersonnel*.

2.1.2 Acteurs systèmes

- **ERP / fichier Excel** : source du stock théorique (colonnes CodeEmplacement, CodeUS, Qte, CodeAgent).
- **SQL Server** : persistance centralisée (base VectorysDB).
- **Hub SignalR** : diffusion temps réel des événements *NouveauScan* et *FinComptage*.
- **SQLite mobile** : cache local pour le mode hors ligne.

2.2 Analyse des besoins fonctionnels

Les besoins fonctionnels ont été regroupés par domaine métier. Chaque exigence est identifiée par un code RF-xx et associée à au moins un acteur.

2.2.1 Authentification et gestion des utilisateurs

ID	Exigence	Acteur	Priorité
RF-01	Connexion agent par code personnel avec émission JWT	Agent	Haute
RF-02	Connexion superviseur/admin par email et mot de passe	Superviseur	Haute
RF-03	Contrôle d'accès par rôle sur chaque endpoint API	Système	Haute
RF-04	Rejet des comptes inactifs (Actif = false)	Système	Moyenne

2.2.2 Gestion des inventaires et passages

ID	Exigence	Acteur	Priorité
RF-05	Créer un inventaire avec référence auto INV-{année}-{seq}	Superviseur	Haute
RF-06	Créer automatiquement le passage C1 à la création	Système	Haute
RF-07	Lister les inventaires avec pagination	Superviseur	Moyenne
RF-08	Créer un passage C2 pour recomptage	Superviseur	Haute
RF-09	Suivre l'avancement temps réel par agent et emplacement	Superviseur	Haute

2.2.3 Import Excel et assignation

ID	Exigence	Acteur	Priorité
RF-10	Importer un fichier Excel (.xlsx) sur un passage	Superviseur	Haute

RF-11	Valider les colonnes obligatoires dont CodeAgent	Système	Haute
RF-12	Créer emplacements, lignes StockThéorique et assignations	Système	Haute
RF-13	Produire un rapport d'import par agent	Superviseur	Moyenne
RF-14	Passer l'inventaire au statut EN_COURS après import réussi	Système	Haute

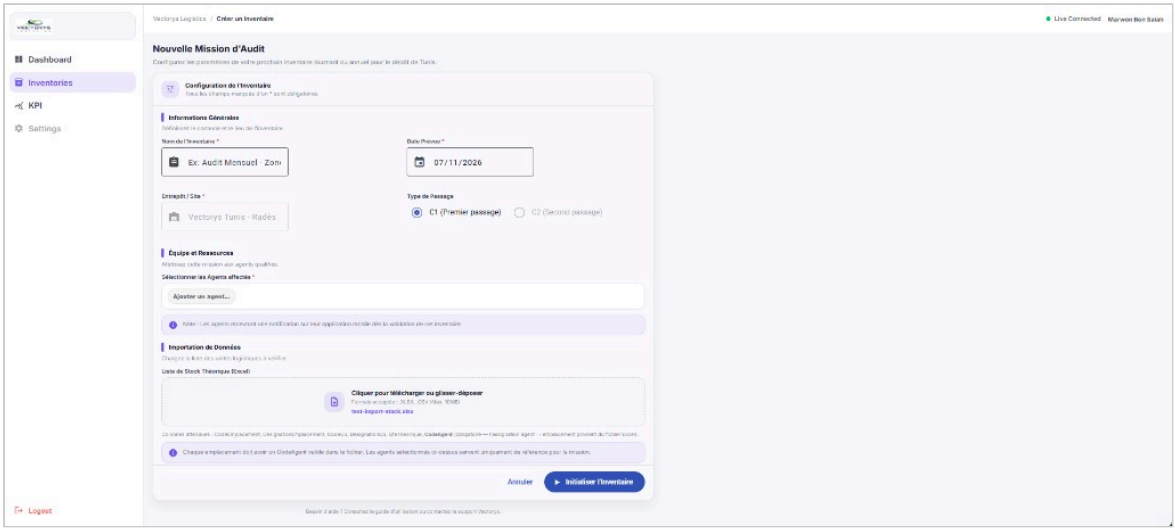
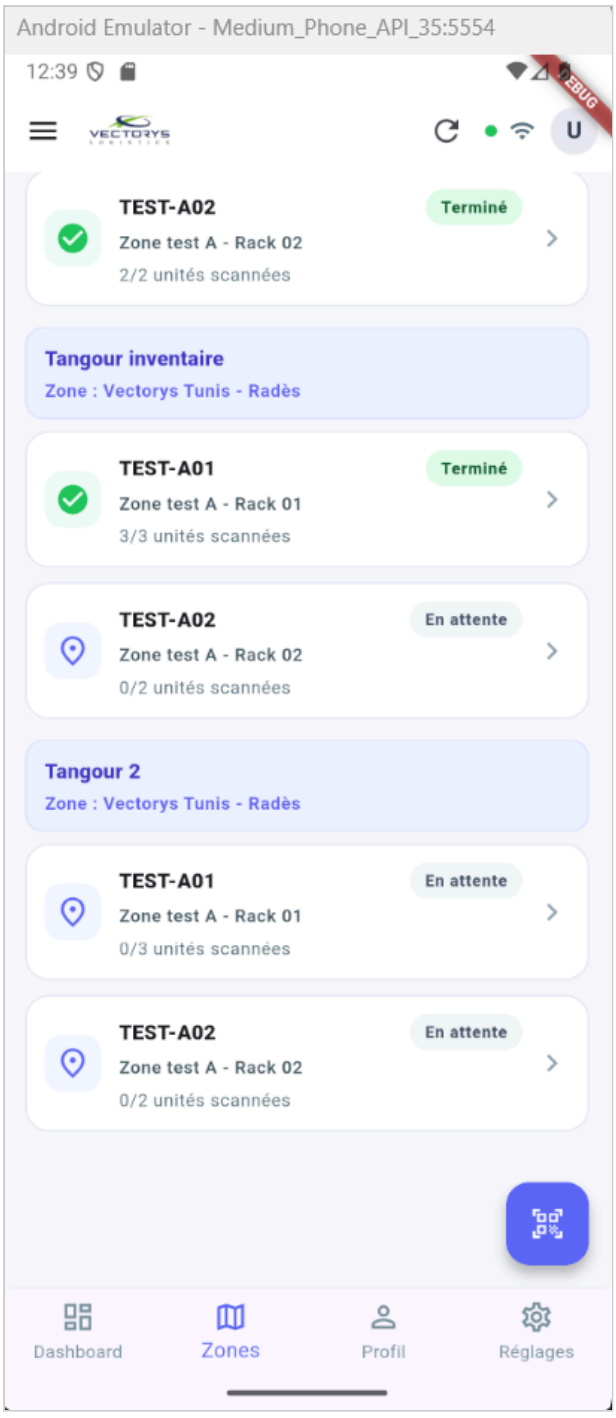
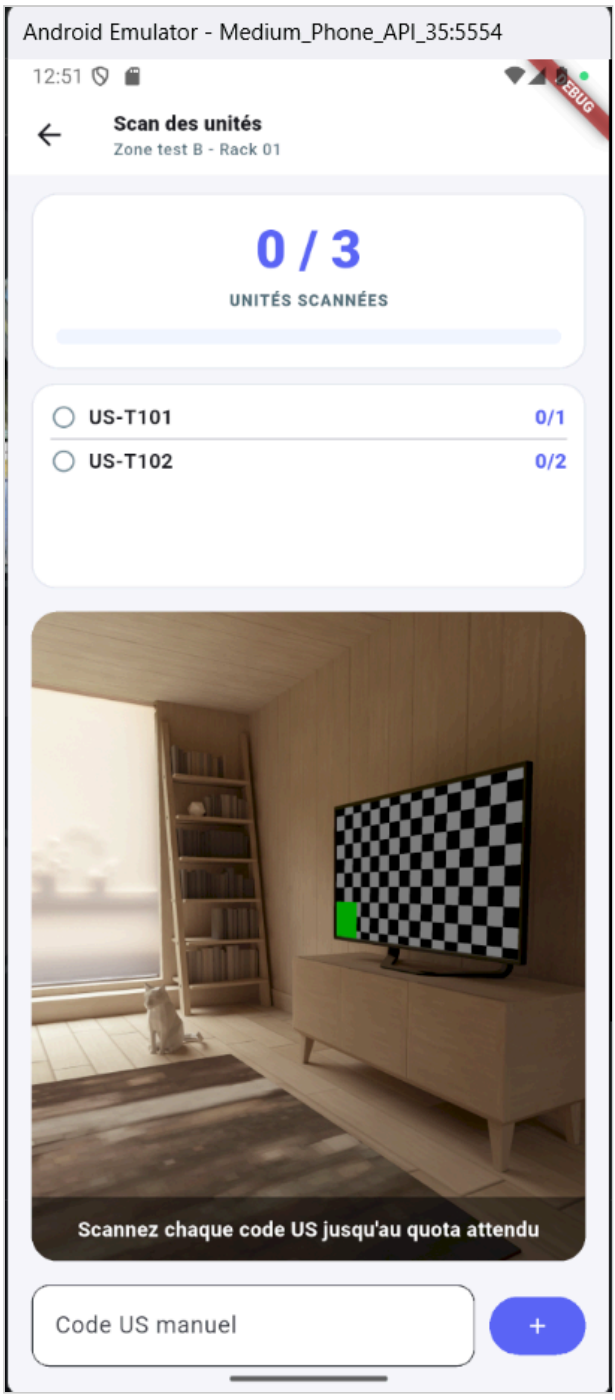


Figure 2.1 — Écran de création d'inventaire et import du stock théorique (interface superviseur Angular)

2.2.4 Comptage terrain (scan)

ID	Exigence	Acteur	Priorité
RF-15	Afficher uniquement les emplacements assignés à l'agent connecté	Agent	Haute
RF-16	Enregistrer un scan de code US avec horodatage	Agent	Haute
RF-17	Refuser les scans au-delà du quota QteThéorique par US	Système	Haute
RF-18	Clôturer automatiquement l'emplacement (TERMINE) quand tous les US sont couverts	Système	Haute
RF-19	Afficher la progression par US (scanné / théorique)	Agent	Haute
RF-20	Synchroniser les scans offline par batch sans doublon (IdLocalFlutter)	Agent	Haute


Android Emulator - Medium_Phone_API_35:5554
12:39
DEBUG
VECTORYS
U
TEST-A02
Terminé
Zone test A - Rack 02
2/2 unités scannées
Tangour inventaire
Zone : Vectorys Tunis - Radès
TEST-A01
Terminé
Zone test A - Rack 01
3/3 unités scannées
TEST-A02
En attente
Zone test A - Rack 02
0/2 unités scannées
Tangour 2
Zone : Vectorys Tunis - Radès
TEST-A01
En attente
Zone test A - Rack 01
0/3 unités scannées
TEST-A02
En attente
Zone test A - Rack 02
0/2 unités scannées
Dashboard
Zones
Profil
Réglages


Android Emulator - Medium_Phone_API_35:5554
12:51
DEBUG
Scan des unités
Zone test B - Rack 01
0 / 3
UNITÉS SCANNÉES
US-T101
0/1
US-T102
0/2
Scannez chaque code US jusqu'au quota attendu
Code US manuel
+

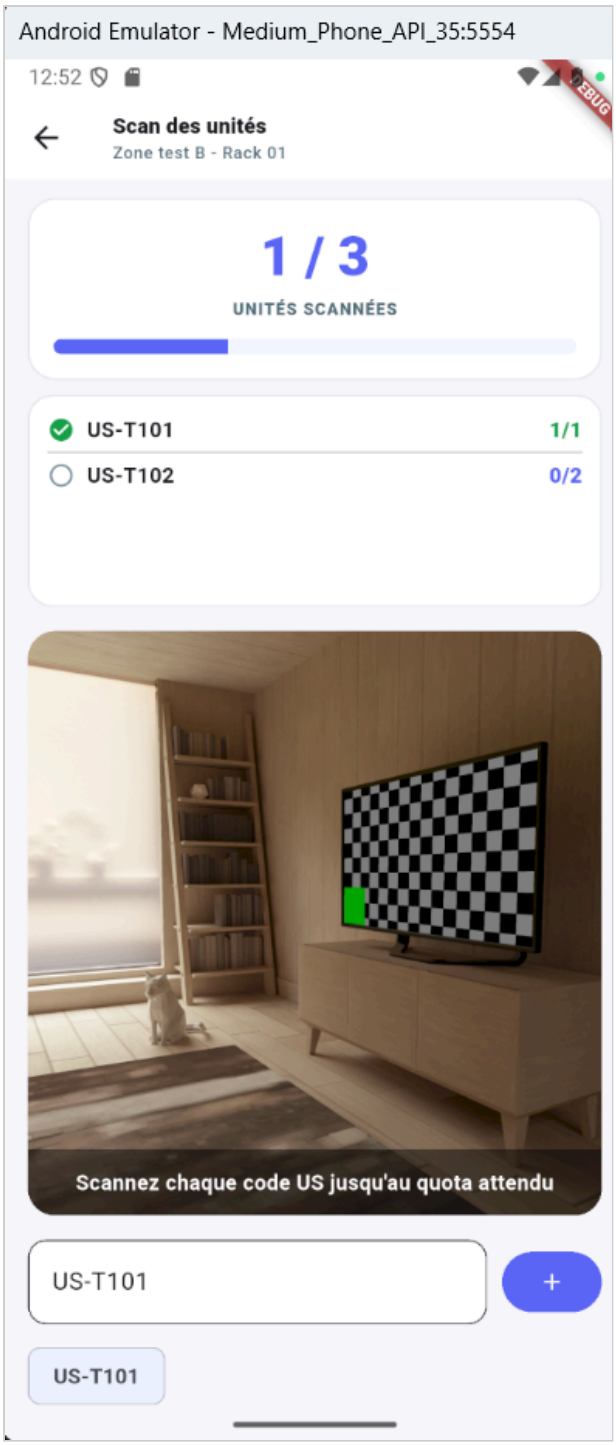


Figure 2.2 — Interface agent : liste des zones, scan en boucle et progression par unité de stock (application mobile Flutter)

2.2.5 Gestion des écarts

ID	Exigence	Acteur	Priorité
RF-21	Détecter les écarts MANQUANT et SURPLUS sur emplacements TERMINE	Superviseur	Haute
RF-22	Lister les écarts par inventaire avec agrégats	Superviseur	Haute

RF-23	Appliquer la branche A : marquer écarts traités, créer C2, cloner emplacements	Superviseur	Haute
RF-24	Supporter la branche B (recomptage global et fusion)	Superviseur	Moyenne

2.2.6 Pilotage KPI et temps réel

ID	Exigence	Acteur	Priorité
RF-25	Notifier le superviseur en temps réel (SignalR) à chaque scan et clôture	Superviseur	Haute
RF-26	Consulter les KPI précision, performance, qualité, clients, WMS	Superviseur	Haute
RF-27	Filtrer les KPI par inventaire, période et zone	Superviseur	Moyenne

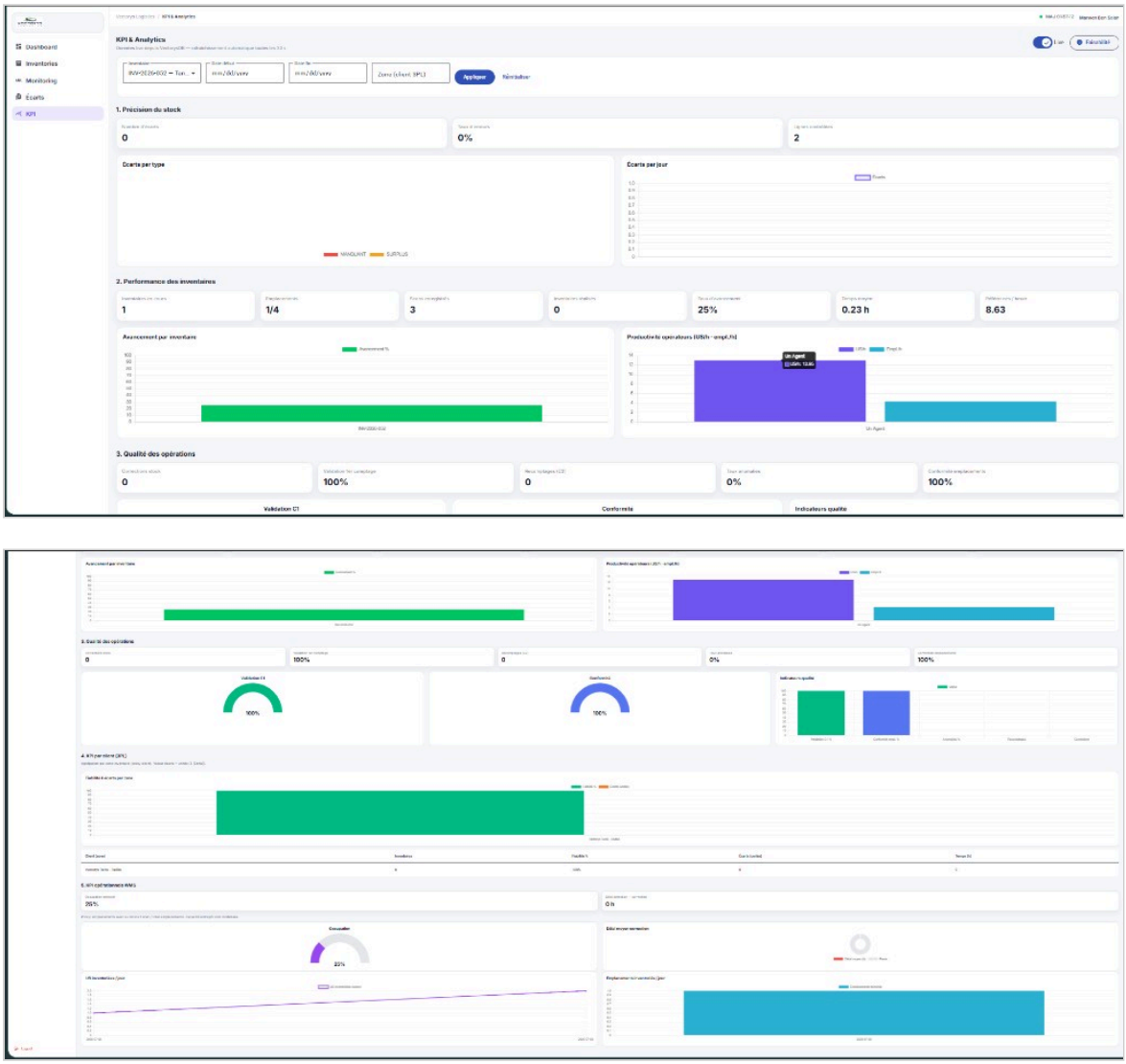


Figure 2.3 — Dashboard KPI : précision, performance, qualité, clients et indicateurs WMS (interface superviseur Angular)

2.3 Besoins non fonctionnels

Les exigences non fonctionnelles garantissent la qualité d'exploitation de la plateforme en environnement industriel.

ID	Catégorie	Exigence	Solution retenue
RNF-01	Sécurité	Authentification et autorisation robustes	JWT HMAC-SHA256, policies SuperviseurOnly / AgentOnly, BCrypt
RNF-02	Performance	Import Excel de milliers de lignes	Insertion par batch (1000 lignes), limite upload 50 Mo
RNF-03	Disponibilité	Continuité en zone Wi-Fi faible	SQLite local + sync batch au retour réseau
RNF-04	Intégrité	Pas de doublons offline	Index unique IdLocalFlutter, idempotence API
RNF-05	Temps réel	Latence faible superviseur	SignalR, groupes par inventaireId
RNF-06	Maintenabilité	Séparation des responsabilités	Clean Architecture, CQRS MediatR, FluentValidation
RNF-07	Observabilité	Traçabilité des erreurs	Serilog, middleware ExceptionHandlingMiddleware
RNF-08	Interopérabilité	API documentée	Swagger OpenAPI, JSON camelCase
RNF-09	Ergonomie	Scan rapide terrain	Boucle scan Flutter, retour visuel/haptique, quotas
RNF-10	Scalabilité KPI	Requêtes agrégées performantes	Index SQL Sprint 7, vues KPI optionnelles

2.4 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation UML ci-dessous modélise l'ensemble des interactions entre les acteurs identifiés (section 2.1) et le système Vectorys Inventaire. Il couvre les trente-deux cas d'utilisation (UC-01 à UC-32) regroupés en sept packages fonctionnels, en correspondance directe avec les exigences RF-01 à RF-27 et les endpoints API implémentés. L'administrateur hérite par généralisation des cas du superviseur ; l'agent n'accède qu'aux cas de comptage terrain.

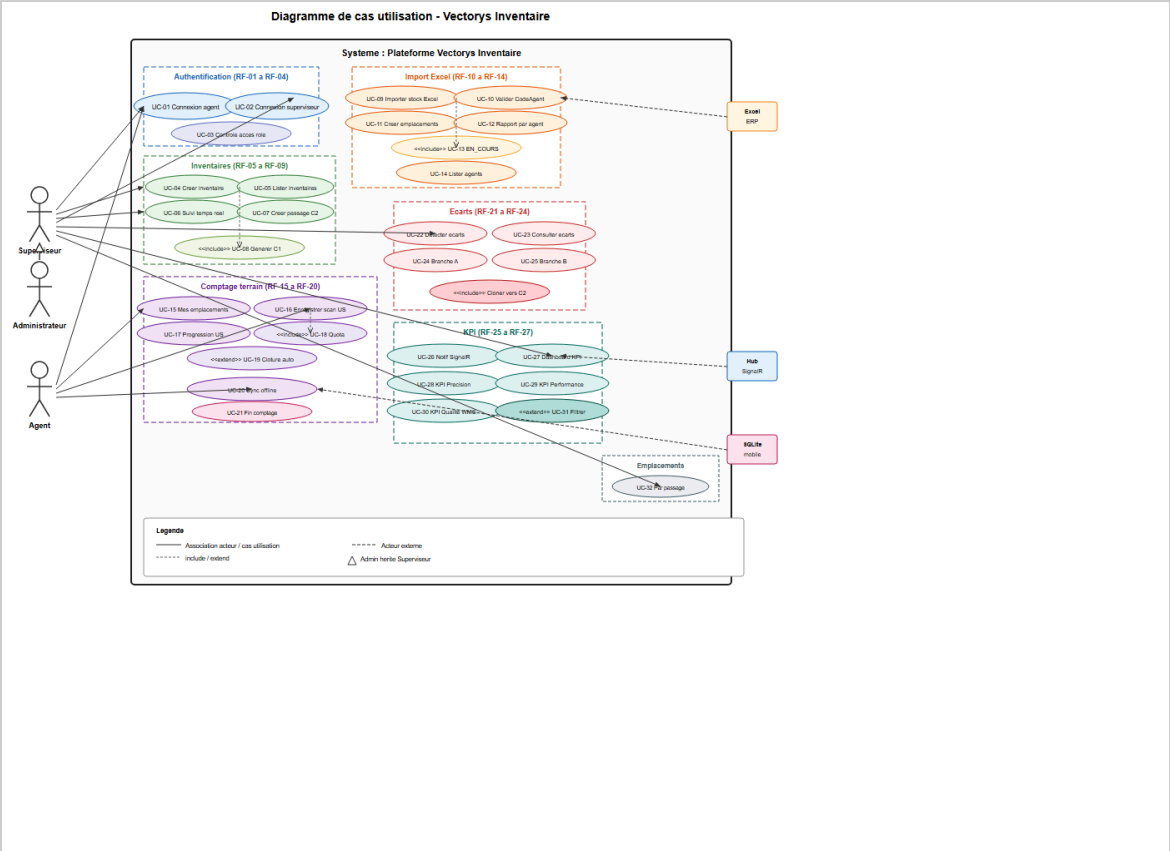


Figure 2.4 — Diagramme de cas d'utilisation détaillé de la plateforme Vectorys Inventaire

2.4.1 Catalogue des cas d'utilisation

Le tableau suivant récapitule les cas d'utilisation principaux, leurs acteurs et les exigences associées.

ID	Cas d'utilisation	Acteur(s)	Exigence	API / composant
UC-01	Connexion agent (JWT 8h)	Agent	RF-01	POST /api/auth/login
UC-02	Connexion superviseur/admin (JWT 24h)	Superviseur, Admin	RF-02	POST /api/auth/login
UC-03	Contrôle d'accès par rôle	Système	RF-03, RF-04	Policies JWT
UC-04	Créer un inventaire (+ réf. auto)	Superviseur	RF-05	POST /api/inventaires
UC-05	Lister les inventaires	Superviseur	RF-07	GET /api/inventaires
UC-06	Suivi temps réel par agent	Superviseur	RF-09	GET /api/inventaires/{id}/suivi-temps-reel

UC-07	Créer passage C2	Superviseur	RF-08	POST /api/passages
UC-08	Générer passage C1 automatiquement	Système	RF-06	<<include>> UC-04
UC-09	Importer stock théorique Excel	Superviseur, Excel	RF-10	POST /api/passages/{id}/import-stock
UC-10	Valider colonne CodeAgent obligatoire	Système	RF-11	StockImportService
UC-11	Créer emplacements et stock théorique	Système	RF-12	ImportStockCommandHandler
UC-12	Consulter rapport d'import par agent	Superviseur	RF-13	ImportStockResultDto
UC-13	Passer inventaire EN_COURS	Système	RF-14	<<include>> UC-09
UC-14	Lister agents actifs	Superviseur	—	GET /api/agents
UC-15	Consulter mes emplacements assignés	Agent	RF-15	GET /api/emplacements/mes-emplacements
UC-16	Enregistrer scan code US	Agent	RF-16	POST /api/scans
UC-17	Afficher progression par US	Agent	RF-19	GetMesEmplacements + scan loop
UC-18	Vérifier quota QteThéorique	Système	RF-17	ScanCompletionService
UC-19	Clôturer emplacement (TERMINE) auto	Système	RF-18	<<extend>> UC-16
UC-20	Synchroniser scans offline (batch)	Agent, SQLite	RF-20	POST /api/scans/batch
UC-21	Terminer comptage emplacement	Agent	—	PUT /api/scans/{id}/fin-comptage
UC-22	Détecter écarts MANQUANT/SURPLUS	Superviseur	RF-21	POST /api/ecarts/detecter/{id}
UC-23	Consulter liste des écarts	Superviseur	RF-22	GET /api/ecarts
UC-24	Appliquer branche A (→ C2)	Superviseur	RF-23	POST /api/ecarts/branche-a

UC-25	Appliquer branche B (recomptage)	Superviseur	RF-24	Workflow C2 global
UC-26	Recevoir notifications SignalR	Superviseur, Hub	RF-25	/hubs/inventaire
UC-27	Consulter dashboard KPI agrégé	Superviseur	RF-26	GET /api/kpi/dashboard
UC-28	KPI Précision (écarts, taux)	Superviseur	RF-26	GET /api/kpi/precision
UC-29	KPI Performance (avancement, productivité)	Superviseur	RF-26	GET /api/kpi/performance
UC-30	KPI Qualité, clients, WMS	Superviseur	RF-26	GET /api/kpi/qualite, /clients, /wms
UC-31	Filtrer KPI (inventaire, période, zone)	Superviseur	RF-27	<<extend>> UC-27
UC-32	Consulter emplacements par passage	Superviseur	—	GET /api/emplacements

2.4.2 Relations include et extend

- **<<include>>** : UC-04 inclut UC-08 (génération C1) ; UC-09 inclut UC-10, UC-11 et UC-13 ; UC-16 inclut UC-18 (vérification quota) ; UC-27 inclut UC-28 à UC-30.
- **<<extend>>** : UC-19 étend UC-16 lorsque le quota de tous les US est atteint ; UC-31 étend UC-27 lors de l'application de filtres.
- **Généralisation** : l'administrateur hérite de tous les cas d'utilisation du superviseur, avec des droits étendus sur la gestion des comptes.

2.5 Scénario principal de comptage C1

Le scénario nominal ci-dessous décrit le déroulement type d'un inventaire tel qu'il a été implémenté et validé sur la plateforme. Les figures 2.1 à 2.3 illustrent respectivement la préparation superviseur, le comptage agent et le pilotage KPI.

1. Le superviseur crée un inventaire ; le passage C1 est généré automatiquement.
2. Il importe le fichier Excel : emplacements, stock théorique et CodeAgent sont validés.
3. L'inventaire passe EN_COURS ; les agents voient leurs emplacements sur mobile.

4. Chaque scan est enregistré ; l'emplacement passe TERMINE automatiquement au quota complet.
5. Le superviseur reçoit les notifications SignalR et consulte le suivi temps réel.
6. À la fin du C1, il lance la détection d'écarts et applique la branche A si nécessaire.

2.6 Backlog produit et user stories

Le backlog produit a été structuré en dix sprints. Le tableau ci-dessous présente les user stories clés livrées (format : En tant que… Je veux… Afin de…).

Sprint	User story (résumé)	Statut
0	En tant que développeur, je veux une solution Clean Architecture avec BDD, afin de poser les fondations.	Livré
1	En tant qu'agent/superviseur, je veux m'authentifier par JWT, afin d'accéder à mon espace sécurisé.	Livré
2	En tant que superviseur, je veux créer un inventaire et importer Excel, afin de préparer le C1.	Livré
3	En tant qu'agent, je veux scanner les US et voir la progression, afin de compter efficacement.	Livré
4	En tant qu'agent, je veux scanner hors ligne, afin de continuer sans réseau.	Livré
5	En tant que superviseur, je veux détecter et traiter les écarts (branche A), afin de lancer le C2.	Livré
6	En tant que superviseur, je veux clôturer et exporter les résultats, afin de finaliser l'inventaire.	Livré
7	En tant que superviseur, je veux un dashboard KPI, afin de piloter la performance.	Livré
8	En tant que développeur, je veux des tests et Swagger, afin de garantir la qualité.	Livré
9	En tant qu'exploitant, je veux déployer sur IIS et APK, afin de mettre en production.	Livré

2.7 Matrice de traçabilité

La matrice suivante assure la couverture des exigences fonctionnelles par sprint et composant.

Exigence	Sprint	API	Web	Mobile

RF-01 à RF-04	1	AuthController	Login	Login
RF-05 à RF-09	2	InventairesController	Création, suivi	—
RF-10 à RF-14	2	PassagesController	Import wizard	—
RF-15 à RF-19	3	ScansController	SignalR	Scan loop
RF-20	4	Scans/batch	—	Offline sync
RF-21 à RF-24	5–6	EcartsController	Écarts UI	—
RF-25 à RF-27	7	KpiController, Hub	Dashboard KPI	—

Conclusion

Ce chapitre a formalisé les acteurs, les besoins fonctionnels (RF-01 à RF-27) et non fonctionnels (RNF-01 à RNF-10) de la plateforme Vectorys Inventaire. Le diagramme de cas d'utilisation (figure 2.4), les captures d'écran des interfaces superviseur et agent, ainsi que la matrice de traçabilité, confirment que l'ensemble des exigences prioritaires a été couvert par les dix sprints réalisés. Le chapitre 3 détaillera la conception architecturale, le modèle de données et les diagrammes UML qui concrétisent ces spécifications.